SkillSwap

Author information

- **Name: Maghan Das**
- **Neptun code: CCQ946**
- **Date: 05-03-2026**

Project overview

Topic

- SkillSwap – Community Skill Exchange Platform

Short description

- SkillSwap is a web platform where users can offer and learn skills from each other. Students can book sessions with teachers, leave reviews, and explore skills across different categories. Administrators manage users, skills, and reviews to ensure a secure and organized system.

Domain-specific vocabulary

List important terms used in the domain of your application.

- **Skill:** A teachable ability (e.g., Guitar, Python, Cooking)
- **Session:** Scheduled learning interaction between a teacher and a student
- **User Roles:** Student, Teacher, Admin
- **Review:** Feedback left by a student after a session

Functional requirements

Roles

List all user roles in the system.

Role	Features	Permissions
Admin	Manage users, skills, reviews, view all sessions	Full access to all entities
Teacher	Offer skills, manage sessions, view reviews	CRUD only on own skills & sessions
Student	Browse skills, request sessions, leave reviews	CRUD only for requests & reviews

Features by roles

For each role, list what they can do.

- **Admin:**
  - Add/Edit/Delete users
  - Add/Edit/Delete skills
  - Moderate reviews
  - View all sessions
- **Teacher:**
  - Create/Edit/Delete skills they offer
  - View scheduled sessions
  - Respond to session requests
  - View received reviews
- **Student:**
  - Browse skills
  - Request skill sessions
  - Leave reviews
  - View own booked sessions

Technology stack

- **Backend:** Laravel
- **Auth scaffolding:** Laravel Breeze
- **Database:** SQLite
- **Frontend: :** Blade Templates + Tailwind CSS / Daisy ui
- **Other tools/libraries: Composer, VS Code, Git**

Database schema

Entities and relationships

- **Users** → Students, Teachers, Admins
- **Skills** → Taught skills
- **User_Skills** → Pivot table connecting users ↔ skills (many-to-many)
- **Sessions** → Booked skill sessions (1-to-many relationships: teacher → sessions, student → sessions)
- **Reviews** → Reviews tied to sessions
- **Categories** → Skill categories

Raw DBML description

```
Table users {
  id integer [pk]
  name varchar
  email varchar [unique]
  password varchar
  role enum('student','teacher','admin')
  created_at timestamp
  updated_at timestamp
}

Table skills {
```

```
  id integer [pk]
  name varchar
  description text
  category_id integer [ref: > categories.id]
  difficulty enum('beginner','intermediate','advanced')
  created_at timestamp
  updated_at timestamp
}

Table user_skills {
  user_id integer [ref: > users.id]
  skill_id integer [ref: > skills.id]
  experience_level enum('beginner','intermediate','advanced')
  is_teaching boolean
  created_at timestamp
  [pk: user_id, skill_id]
}

Table sessions {
  id integer [pk]
  teacher_id integer [ref: > users.id]
  student_id integer [ref: > users.id]
  skill_id integer [ref: > skills.id]
  title varchar
  description text
  session_date date
  session_time time
  duration integer
  location varchar
  status enum('pending','confirmed','completed','cancelled')
  created_at timestamp
  updated_at timestamp
}

Table reviews {
  id integer [pk]
```

```
  session_id integer [ref: > sessions.id]
  rating integer
  comment text
  created_at timestamp
  updated_at timestamp
}

Table categories {
  id integer [pk]
  name varchar
  description text
  created_at timestamp
  updated_at timestamp
}
```

Database description by tables and fields

Users Table:

- id → Primary key
- name → User full name
- email → Unique email
- password → Hashed password
- role → Enum('student','teacher','admin')
- created_at / updated_at → Timestamps

Skills Table:

- id → Primary key
- name → Skill name
- description → Skill details
- category_id → FK to categories
- difficulty → Enum

- created_at / updated_at → Timestamps

User_Skills Table (Pivot):

- user_id → FK to users
- skill_id → FK to skills
- experience_level → Enum
- is_teaching → Boolean
- created_at → Timestamp

Sessions Table:

- id → Primary key
- teacher_id → FK to users
- student_id → FK to users
- skill_id → FK to skills
- title → Session title
- description → Session description
- session_date → Date of session
- session_time → Time of session
- duration → Duration in minutes
- location → Session location
- status → Enum('pending','confirmed','completed','cancelled')
- created_at / updated_at → Timestamps

Reviews Table:

- id → Primary key
- session_id → FK to sessions
- rating → Integer
- comment → Text feedback
- created_at / updated_at → Timestamps

Categories Table:

- id → Primary key
- name → Category name
- description → Text description
- created_at / updated_at → Timestamps

Planned sitemap (endpoints for features)

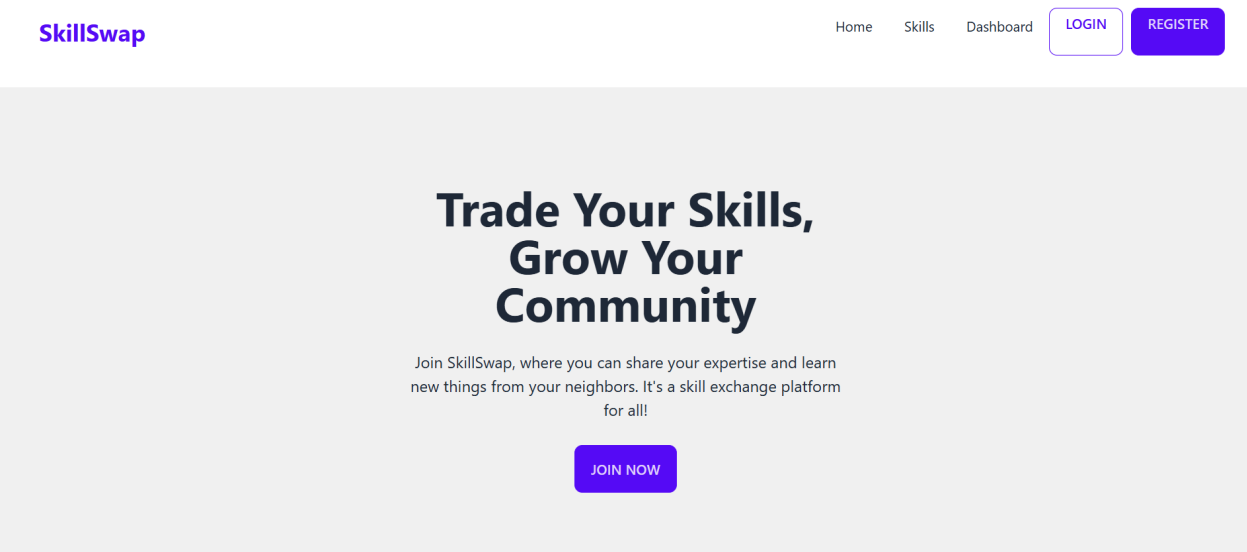
List planned pages and endpoints/routes per feature.

Page	Route / Endpoint		
/	Home Page		
/register	Register		
/login	Login		
/dashboard	User Dashboard		
/skills	skills		
/skills/{id}	skill detail		
/skills/create	Add Skill(Teacher)		
/reviews	reviews details		
/book-session	session booking		
/admin	for admin		

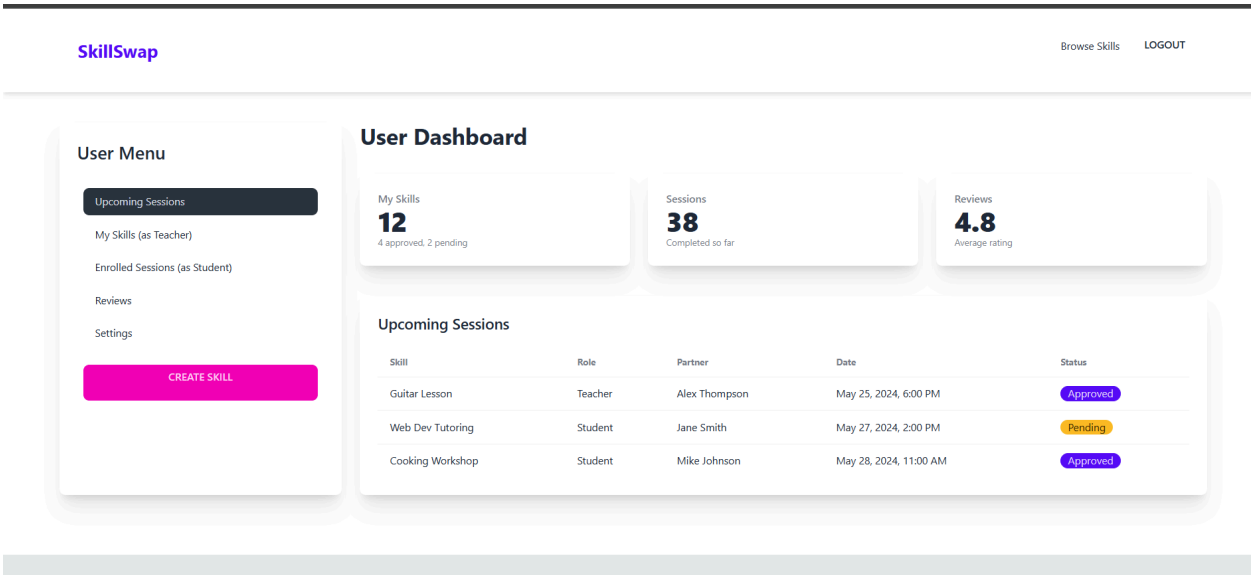
8️⃣ UI Mockups

Suggested mockups:

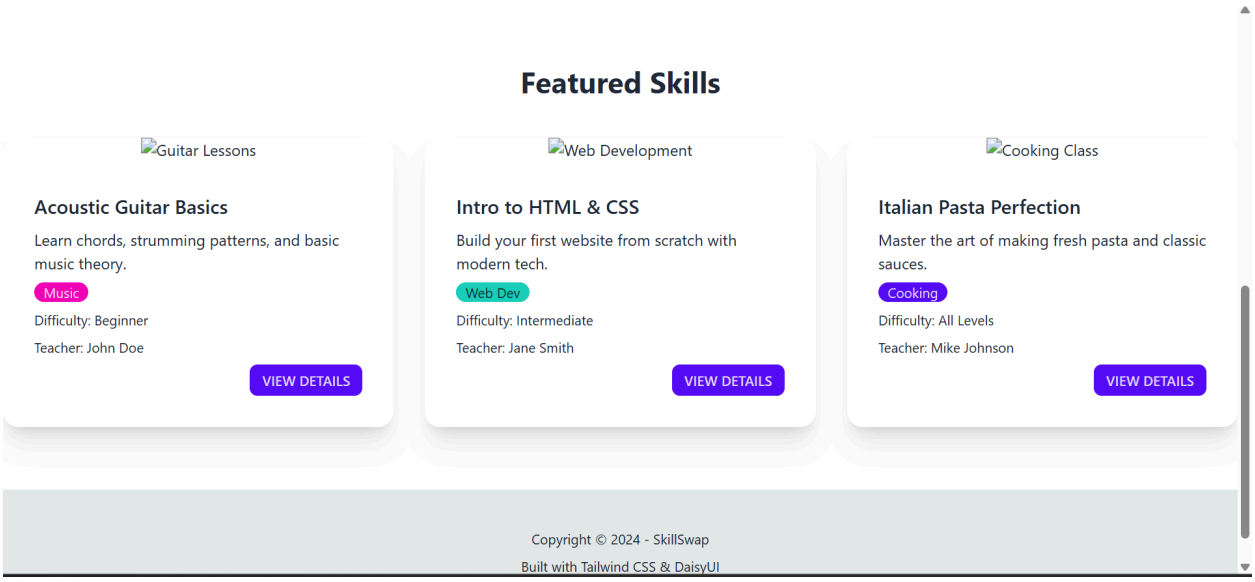
1. Home page



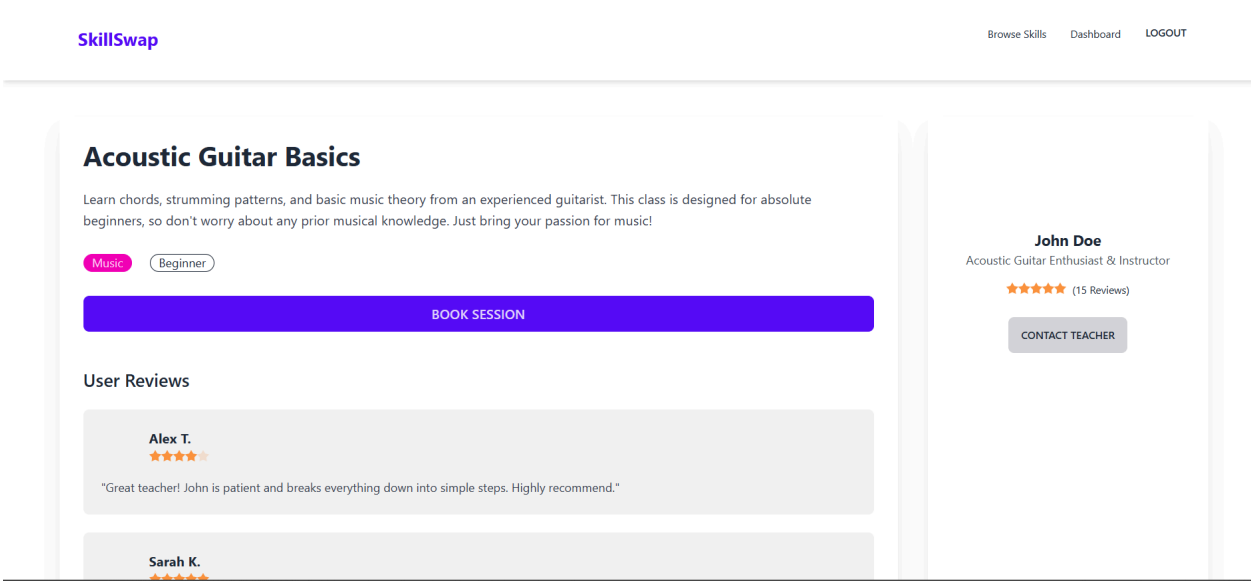
1. User dashboard



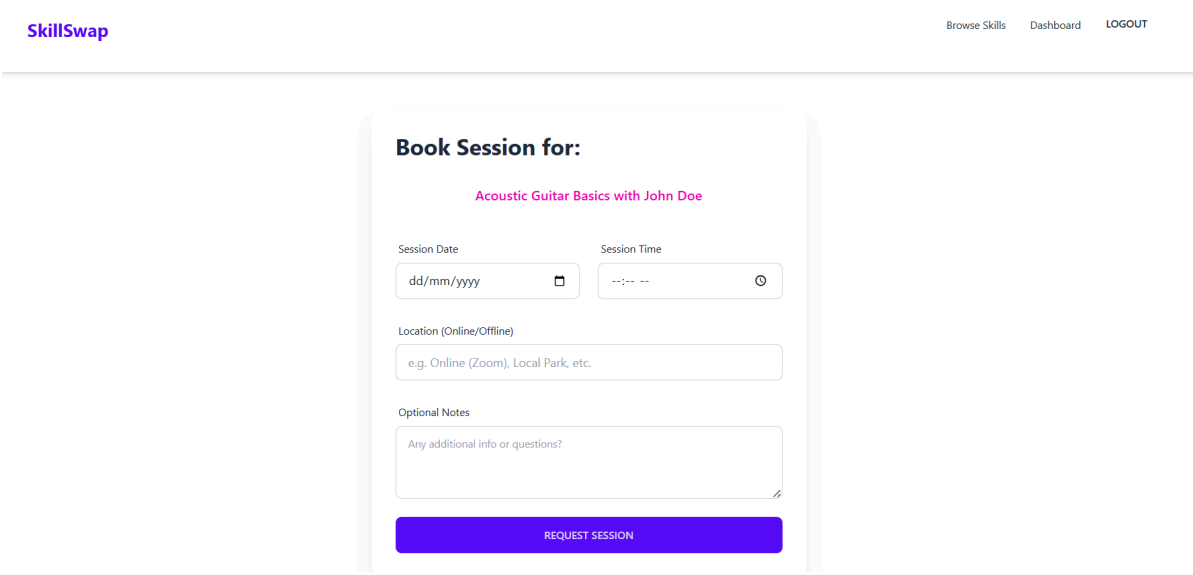
1. Skills list page



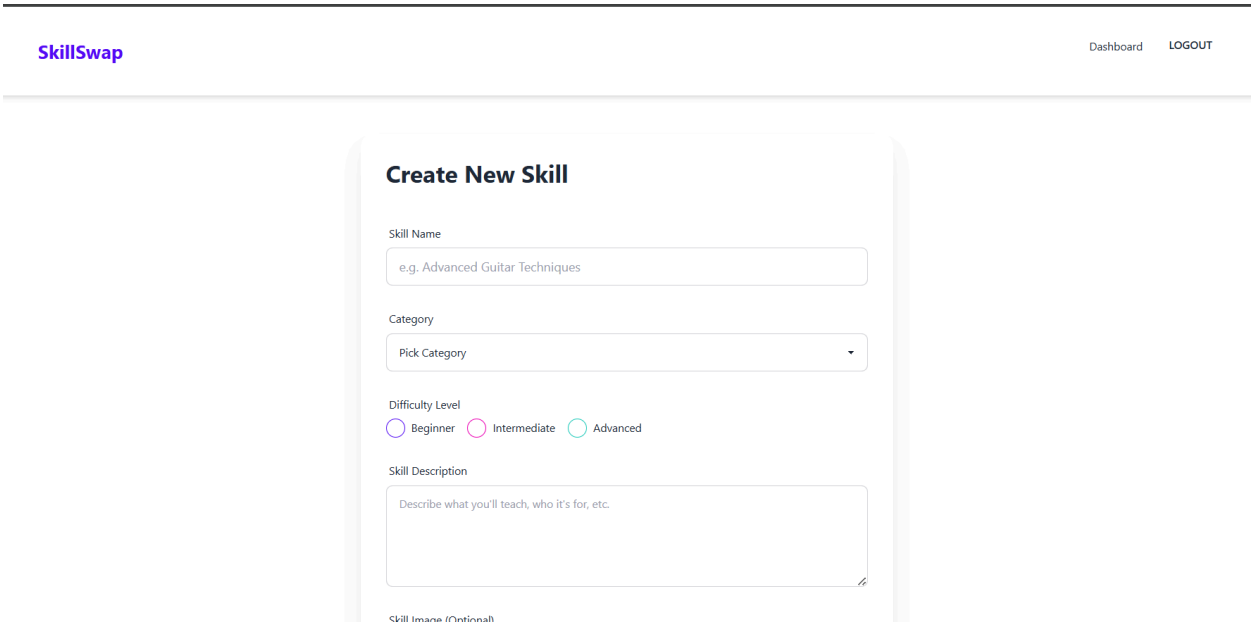
1. Skill detail page with “Book Session” button



1. Session booking form

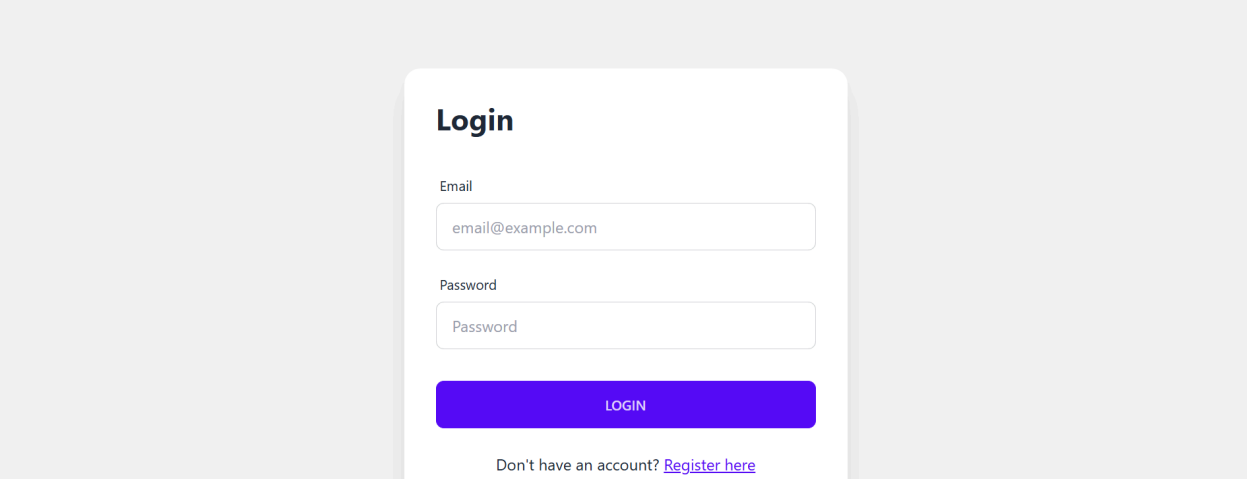


2. create new skill



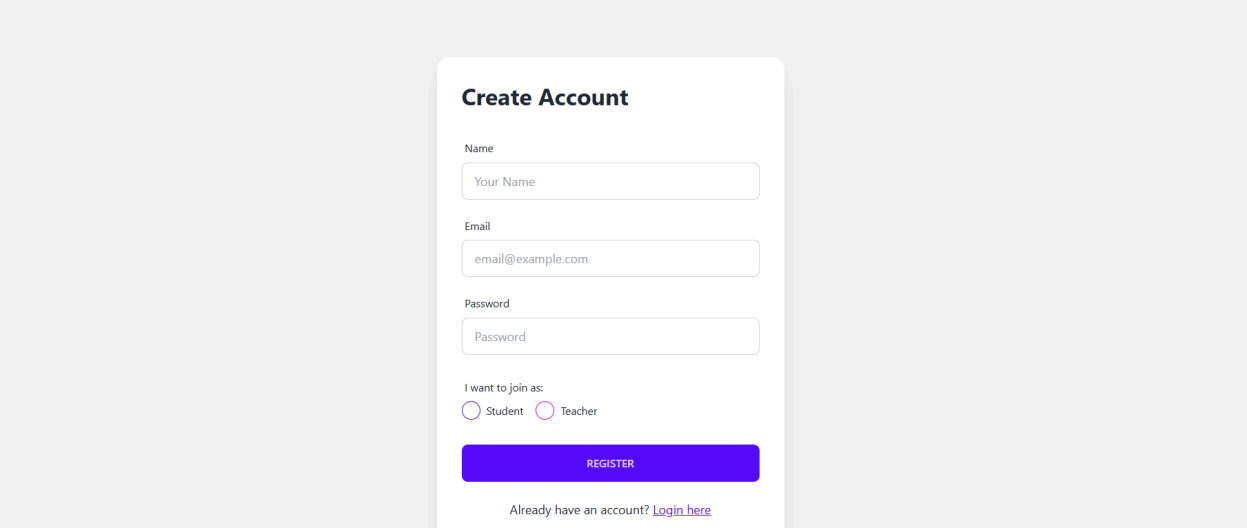
1. Login

SkillSwap

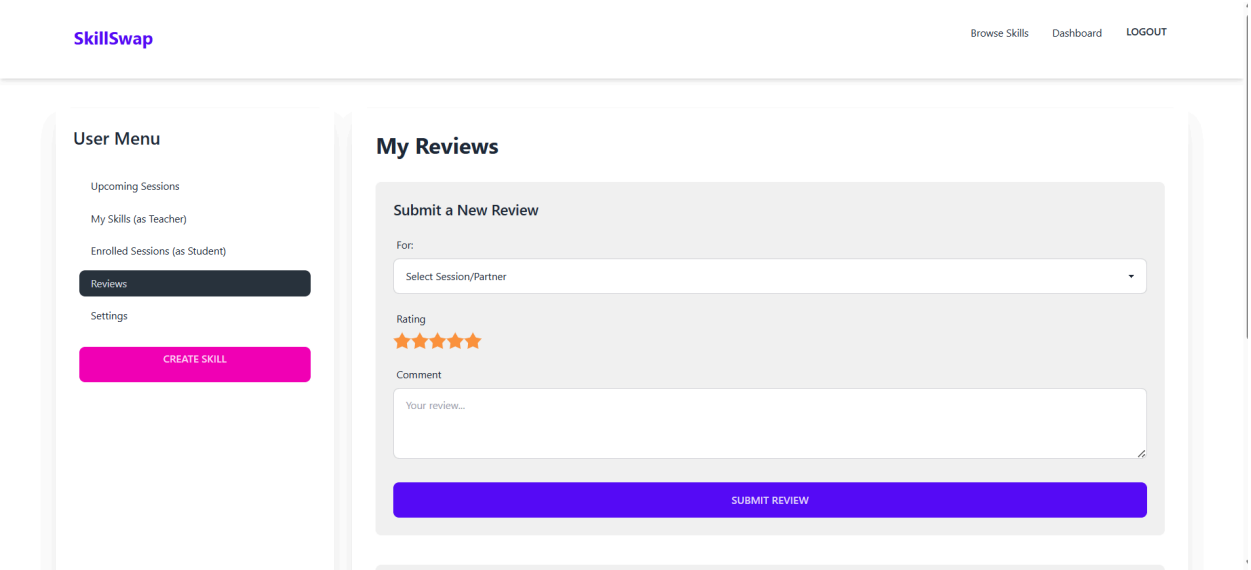


1. Register

SkillSwap



Reviews



AI appendix

Describe any AI usage (if any).

- AI tools were optionally used to **generate database schema**, **draft documentation**, and **suggest UI mockups**.
- The final design, database, and application logic were implemented manually using Laravel.

Phase 1 requirements checklist (copy/paste)

Database requirements

- ☑ ~~At least 5 database tables~~
- ☑ ~~At least one 1-to-N relationship used~~
- ☑ ~~At least one N-to-N relationship with pivot table~~
- ☑ ~~Primary keys, foreign keys, unique constraints used consistently~~
- ☑ ~~At least one table has 10+ fields~~
- ☑ ~~Includes fields with: `integer`, `boolean`, `enum`, `timestamp`, `date`~~

Frontend pages

- [x] ~~Responsive, visually appealing design~~
- [x] ~~Uses a component library~~
- [x] ~~At least 10 distinct pages~~

CRUD functionality

- [x] ~~CRUD for an entity on N side of 1-to-N~~
- [x] ~~CRUD for an entity on N side of N-to-N~~
- [x] ~~Forms include: checkbox list, radio buttons, select dropdown, file upload~~

Authentication & authorization

- [x] ~~Laravel authentication + authorization~~
- [x] ~~Users cannot access other users' protected data~~
- [x] ~~Sensitive actions access-controlled by roles/permissions~~

Admin interfaces

- [x] ~~Admin user management page(s)~~
- [x] ~~Admin CRUD for at least one non-user entity~~

Technology requirements

- [x] ~~Laravel~~
- [x] ~~Laravel Breeze~~
- [x] ~~SQLite~~
